

Advanced Touying

Animation, layout, recall — and one source for two outputs

The Touying Authors

2042-01-01

Since touying **v0.8.0** math is now fully supported.

Marks reach inside math elements

A **#pause** works wherever it is written in an equation — including inside **frac**, **mat** and the other elements that keep their content in named fields of their own, rather than in a **body**:

```
$ frac(a #pause + b, c) $
```

$$\frac{a}{\quad}$$

Marks reach inside math elements

A **#pause** works wherever it is written in an equation — including inside **frac**, **mat** and the other elements that keep their content in named fields of their own, rather than in a **body**:

```
$ frac(a #pause + b, c) $
```

$$\frac{a + b}{c}$$

Marks reach inside math elements

A **#pause** works wherever it is written in an equation — including inside **frac**, **mat** and the other elements that keep their content in named fields of their own, rather than in a **body**:

```
$ frac(a #pause + b, c) $
```

$$\frac{a + b}{c}$$

The same holds for a matrix, cell by cell:

```
$ mat(a, #pause b; c, #pause d) $
```

$$\begin{pmatrix} a \\ b \end{pmatrix}$$

Marks reach inside math elements

A **#pause** works wherever it is written in an equation — including inside **frac**, **mat** and the other elements that keep their content in named fields of their own, rather than in a **body**:

```
$ frac(a #pause + b, c) $
```

$$\frac{a + b}{c}$$

The same holds for a matrix, cell by cell:

```
$ mat(a, #pause b; c, #pause d) $
```

$$\begin{pmatrix} a & b \\ c & \end{pmatrix}$$

Marks reach inside math elements

A **#pause** works wherever it is written in an equation — including inside **frac**, **mat** and the other elements that keep their content in named fields of their own, rather than in a **body**:

```
$ frac(a #pause + b, c) $
```

$$\frac{a + b}{c}$$

The same holds for a matrix, cell by cell:

```
$ mat(a, #pause b; c, #pause d) $
```

$$\begin{pmatrix} a & b \\ c & d \end{pmatrix}$$

Building a derivation

#uncover and **#only** work in math too, so a derivation can grow a term at a time while the layout stays put:

```
$ f(x) &= x^2 #uncover("2-")[$+ 2x$]#uncover("3-")[$+ 1$] \  
#only("3-")[$&= (x + 1)^2$] $
```

$$f(x) = x^2$$

Building a derivation

#uncover and **#only** work in math too, so a derivation can grow a term at a time while the layout stays put:

```
$ f(x) &= x^2 #uncover("2-")[$+ 2x$]#uncover("3-")[$+ 1$] \  
#only("3-")[$&= (x + 1)^2$] $
```

$$f(x) = x^2 + 2x$$

#uncover reserves the space it will occupy, so nothing shifts as terms appear. **#only** removes it, which is why the second line arrives late.

Building a derivation

#uncover and **#only** work in math too, so a derivation can grow a term at a time while the layout stays put:

```
$ f(x) &= x^2 #uncover("2-")[$+ 2x$]#uncover("3-")[$+ 1$] \
    #only("3-")[$&= (x + 1)^2$] $
```

$$\begin{aligned} f(x) &= x^2 + 2x + 1 \\ &= (x + 1)^2 \end{aligned}$$

#uncover reserves the space it will occupy, so nothing shifts as terms appear. **#only** removes it, which is why the second line arrives late.

alternatives and its cousins

`#alternatives` shows one body per subslide, possibly reserving room for the largest with `stretch: true` so the surrounding layout never jumps:

```
#alternatives(stretch: true)[first][second]  
[third]
```

first

alternatives and its cousins

`#alternatives` shows one body per subslide, possibly reserving room for the largest with `stretch: true` so the surrounding layout never jumps:

```
#alternatives(stretch: true)[first][second]  
[third]
```

second

alternatives and its cousins

`#alternatives` shows one body per subslide, possibly reserving room for the largest with `stretch: true` so the surrounding layout never jumps:

```
#alternatives(stretch: true)[first][second]  
[third]
```

third

`#alternatives-cases` is the same idea driven by a function, which e.g. allows writing a list where one row is highlighted at a time. The strings say which subslides each case covers:

```
#alternatives-cases(("3", "4", "5"), case => {  
  let rows = ("chlorophyll", "carotene", "anthocyanin")  
  stack(dir: ltr, spacing: 1.2em,  
    ..rows.enumerate().map((i, r)) => {  
      if i == case { strong(r) } else { text(fill: gray, r) }  
    }),  
  })  
})
```

chlorophyll carotene anthocyanin

alternatives and its cousins

`#alternatives` shows one body per subslide, possibly reserving room for the largest with `stretch: true` so the surrounding layout never jumps:

```
#alternatives(stretch: true)[first][second]
[third]
```

third

`#alternatives-cases` is the same idea driven by a function, which e.g. allows writing a list where one row is highlighted at a time. The strings say which subslides each case covers:

```
#alternatives-cases(("3", "4", "5"), case => {
  let rows = ("chlorophyll", "carotene", "anthocyanin")
  stack(dir: ltr, spacing: 1.2em,
    ..rows.enumerate().map((i, r)) => {
      if i == case { strong(r) } else { text(fill: gray, r) }
    }),)
})
```

chlorophyll **carotene** anthocyanin

alternatives and its cousins

`#alternatives` shows one body per subslide, possibly reserving room for the largest with `stretch: true` so the surrounding layout never jumps:

```
#alternatives(stretch: true)[first][second]  
[third]
```

third

`#alternatives-cases` is the same idea driven by a function, which e.g. allows writing a list where one row is highlighted at a time. The strings say which subslides each case covers:

```
#alternatives-cases(("3", "4", "5"), case => {  
  let rows = ("chlorophyll", "carotene", "anthocyanin")  
  stack(dir: ltr, spacing: 1.2em,  
    ..rows.enumerate().map((i, r)) => {  
      if i == case { strong(r) } else { text(fill: gray, r) }  
    }),  
  })  
})
```

chlorophyll carotene **anthocyanin**

animate composes effects

`#animate` applies several effects to one body, each over its own subslide range. A **placement** decides whether the body is shown, covered or removed; a **styling** function wraps it. The last one written wins a tie but you can also use **priority**.

There are 4 placements: `"show"`, `"cover"`, `"remove"` and `swap()`. The default effect uses the placement `"show"` with subslides `"1-"` and priority `0` which is why “Photosynthesis” shows on the first subslide.

```
#animate(  
  [Photosynthesis],  
  effects: (  
    (effect: swap([Respiration], stretch: true), subslides: "2-"),  
    (effect: (body, self: none) => text(fill: red, body), subslides: "3-"),  
    (effect: swap([Photosynthesis], stretch: true), subslides: 4, priority: 2)  
  ),  
)
```

Photosynthesis

animate composes effects

`#animate` applies several effects to one body, each over its own subslide range. A **placement** decides whether the body is shown, covered or removed; a **styling** function wraps it. The last one written wins a tie but you can also use **priority**.

There are 4 placements: `"show"`, `"cover"`, `"remove"` and `swap()`. The default effect uses the placement `"show"` with subslides `"1-"` and priority `0` which is why “Photosynthesis” shows on the first subslide.

```
#animate(  
  [Photosynthesis],  
  effects: (  
    (effect: swap([Respiration], stretch: true), subslides: "2-"),  
    (effect: (body, self: none) => text(fill: red, body), subslides: "3-"),  
    (effect: swap([Photosynthesis], stretch: true), subslides: 4, priority: 2)  
  ),  
)
```

Respiration

animate composes effects

`#animate` applies several effects to one body, each over its own subslide range. A **placement** decides whether the body is shown, covered or removed; a **styling** function wraps it. The last one written wins a tie but you can also use **priority**.

There are 4 placements: `"show"`, `"cover"`, `"remove"` and `swap()`. The default effect uses the placement `"show"` with subslides `"1-"` and priority `0` which is why “Photosynthesis” shows on the first subslide.

```
#animate(  
  [Photosynthesis],  
  effects: (  
    (effect: swap([Respiration], stretch: true), subslides: "2-"),  
    (effect: (body, self: none) => text(fill: red, body), subslides: "3-"),  
    (effect: swap([Photosynthesis], stretch: true), subslides: 4, priority: 2)  
  ),  
)
```

Respiration

animate composes effects

`#animate` applies several effects to one body, each over its own subslide range. A **placement** decides whether the body is shown, covered or removed; a **styling** function wraps it. The last one written wins a tie but you can also use **priority**.

There are 4 placements: `"show"`, `"cover"`, `"remove"` and `swap()`. The default effect uses the placement `"show"` with subslides `"1-"` and priority `0` which is why “Photosynthesis” shows on the first subslide.

```
#animate(  
  [Photosynthesis],  
  effects: (  
    (effect: swap([Respiration], stretch: true), subslides: "2-"),  
    (effect: (body, self: none) => text(fill: red, body), subslides: "3-"),  
    (effect: swap([Photosynthesis], stretch: true), subslides: 4, priority: 2)  
  ),  
)
```

Photosynthesis

Waypoints name positions

Counting subslides breaks the moment you insert a **#pause** above. A waypoint names a position instead, and everything referring to it follows:

```
#waypoint(<detail>, advance:false)
#uncover(<detail>)[This line names `<detail>`, not "subslide 2".]
#pause
#only(get-last(<detail>))[`#get-last` pins the waypoint's final subslide – useful
when a range should end exactly where a phase does.]
#waypoint(<other-stuff>)
Waypoints capture all subslides up until the next waypoint.
```

Numbers stay correct here no matter what is added before the waypoint.

Waypoints name positions

This line names `<detail>`, not “subslide 2”.

Waypoints name positions

This line names `<detail>`, not “subslide 2”.

`#get-last` pins the waypoint’s final subslide — useful when a range should end exactly where a phase does.

Waypoints name positions

Waypoints capture all subslides up until the next waypoint.

Recalling a slide

A slide can carry a label either via the heading or the slide function, and `#touying-recall` brings it back later — handy for a summary that should show the finished state of an earlier build:

```
== Shared heading  
#slide[.. first ..]<intro-a>  
#slide[.. second ..]<intro-b>  
  
#touying-recall(<intro-b>)
```

Recalling a slide

A slide can carry a label either via the heading or the slide function, and `#touying-recall` brings it back later — handy for a summary that should show the finished state of an earlier build:

```
== Shared heading  
#slide[.. first ..]<intro-a>  
#slide[.. second ..]<intro-b>  
  
#touying-recall(<intro-b>)
```

You may also pass in a subslide index, range or waypoint to recall whatever stage you want to show again.

Rendering Content

You may also store some content in a variable and render only subslices of it via `#touying-render`.

```
#let my-table = table(  
  columns: 2,  
  [Header 1], [Header 2],  
  pause, [Cell 1],  
  [Cell 2], pause,  
  [Cell 3], [Cell 4],  
)  
  
#touying-render(my-table, subslices: 2)
```

Header 1	Header 2
Cell 1	Cell 2

Rendering Content

You may also store some content in a variable and render only subslices of it via `#touying-render`.

```
#let my-table = table(  
  columns: 2,  
  [Header 1], [Header 2],  
  pause, [Cell 1],  
  [Cell 2], pause,  
  [Cell 3], [Cell 4],  
)  
  
#touying-render(my-table, subslices: 2)
```

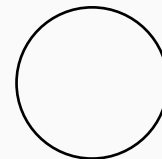
Header 1	Header 2
Cell 1	Cell 2

It does not have to be a single element but can be even a reveal sequence leaving individual steps out.

Animating a drawing

`#touying-reduce` wires an external drawing package into the same animation system, so `pause` inside a `cetz.canvas` means what it means everywhere else:

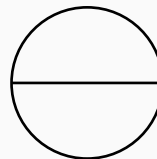
```
#let cetz-canvas = touying-  
reduce.with(cetz)  
  
#cetz-canvas({  
  import cetz.draw: *  
  circle((0, 0), radius: 1)  
  (pause,)   
  line((-1, 0), (1, 0))  
})
```



Animating a drawing

`#touying-reduce` wires an external drawing package into the same animation system, so `pause` inside a `cetz.canvas` means what it means everywhere else:

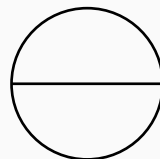
```
#let cetz-canvas = touying-  
reduce.with(cetz)  
  
#cetz-canvas({  
  import cetz.draw: *  
  circle((0, 0), radius: 1)  
  (pause,)   
  line((-1, 0), (1, 0))  
})
```



Animating a drawing

`#touying-reduce` wires an external drawing package into the same animation system, so `pause` inside a `cetz.canvas` means what it means everywhere else:

```
#let cetz-canvas = touying-  
reduce.with(cetz)  
  
#cetz-canvas({  
  import cetz.draw: *  
  circle((0, 0), radius: 1)  
  (pause,)  
  line((-1, 0), (1, 0))  
})
```



Touying supplies¹ the package's own cover function, so a hidden part keeps its size and the drawing does not jump as it is revealed.

You may also use the synonym `touying-diagram` instead of `touying-reduce`.

¹For Cetz, Fletcher and Alchemist we supply the bindings. Other packages need to expose those to touying themselves.

Two columns, and what the article does with them

A slide can be split into multiple columns via

```
#slide[left][right]
```

or by passing in an explicit **composer** to split the columns differently:

```
#slide(composer:(2fr, 1fr))[left][right]
```

Two columns, and what the article does with them

A slide can be split into multiple columns via

```
#slide[left][right]
```

or by passing in an explicit `composer` to split the columns differently:

```
#slide(composer:(2fr, 1fr))[left][right]
```

An article has no slide to divide, so a composer's columns are flattened back into running text. That is usually what you want when the same source has to read as prose — and when it is not, `#article-keep-layout[...]` holds a container together: usable on columns, grids, tables. Tables and grids are by default linearized if they have no heading or footer. Read more in the docs on how to configure this.

Selecting what goes where.

Five markers decide where content goes. `#presentation-only[...]`, `#handout-only[...]` and `#article-only[...]` keep content to one output; `#slides-only[...]` routes it to both the presentation and handout. `#article-text[...]` replaces a slide's body with prose written for reading, which is what nearly every slide in this deck does. The same filtering of `...only` is available as `<touying:...>` labels on a heading or a slide

Selecting what goes where.

see Output Modes.

Selecting what goes where.

see Output Modes.

You are reading the Presentation, so this paragraph — written inside `#slides-only[..]` — is visible and the article's paragraph is not.

Selecting what goes where.

see Output Modes.

You are reading the Presentation, so this paragraph — written inside `#slides-only[...]` — is visible and the article's paragraph is not.

The rule of thumb: write the slide for projection, then add `#article-text[...]` wherever the prose version needs to differ. It replaces the entire slide content with written prose. See the docs or this source.

Handout mode

A handout is a slide output with the animation flattened: every slide keeps by default only its last subslide, so a reader gets one page per slide instead of one per reveal. Compile it with `--input export-mode=handout` or set `handout: true` inside `config-common`.

Handout mode

A handout is a slide output with the animation flattened: every slide keeps by default only its last subslide, so a reader gets one page per slide instead of one per reveal. Compile it with `--input export-mode=handout` or set `handout: true` inside `config-common`.

`#presentation-only[...]` and `#handout-only[...]` split content between the two, which is how a deck carries speaker-facing and reader-facing wording in one file:

You are reading the live presentation. This paragraph sits inside `#presentation-only[...]`, so the handout shows the other one instead.

Handout mode

A handout is a slide output with the animation flattened: every slide keeps by default only its last subslide, so a reader gets one page per slide instead of one per reveal. Compile it with `--input export-mode=handout` or set `handout: true` inside `config-common`.

`#presentation-only[..]` and `#handout-only[..]` split content between the two, which is how a deck carries speaker-facing and reader-facing wording in one file:

You are reading the live presentation. This paragraph sits inside `#presentation-only[..]`, so the handout shows the other one instead.

Per-slide control which subslides end up in the handout is available too: `config-common(handout-subslides: ..)` keeps chosen subslides rather than just the last, which is useful when a build only makes sense in stages. Set this via `#tourneying-set-config` or pass it to as an argument to a `#slide` call.